9. sed and awk for Text Engineering

09. sed and awk for Text Engineering

Phase: Phase 3 — Text, pipes, and discovery
Estimated time: 2 days
Slug: `09-sed-awk`

Learning objectives

- Perform common sed substitutions
- Use awk for column processing
- Choose the right tool (cut/sort/uniq vs awk)
- Write small one-liners you can explain

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

When data is a stream of text lines such as logs, CSV-like exports or config files, `sed` and `awk` transform it without opening an editor. `sed` edits streams (substitute, delete, print lines); `awk` treats each line as fields and can compute sums, averages and reports. They appear constantly in CI scripts and incident work.

Core concepts

`sed` reads input line by line into a *pattern space*, applies commands, and prints the result. Its workhorse is substitution: `s/REGEX/REPLACEMENT/FLAGS`. Flags: `g` all occurrences on a line, `i` ignore case, a number replaces only that occurrence. `-n` suppresses automatic printing, `p` prints; `-i` edits files in place (use `-i.bak` to keep a backup).

`awk` runs `pattern { action }` rules per line. Fields are `$1`, `$2`, … split on whitespace by default (`-F,` sets another separator); `$0` is the whole line; `NF` is the number of fields, `NR` the line number. `BEGIN` runs before input, `END` after.

```
input line:   alice  42  admin
               $1    $2   $3      NF=3  NR=1
awk -F: '$3 >= 1000 { print $1 }' /etc/passwd
      |        |            |
 separator  pattern (cond)  action

sed 's/old/new/g'  <  in.txt  >  out.txt
      stream editor: line -> [ pattern space ] -> output
```

Task	sed	awk
Replace text	`sed 's/a/b/g'`	`gsub(/a/,"b")`
Print line 3	`sed -n '3p'`	`NR==3`
Delete blanks	`sed '/^$/d'`	`NF`
Print column 2	awkward	`{print $2}`
Sum a column	not suited	`{s+=$2} END{print s}`

Pick sed for text rewriting and line selection, awk for column logic and arithmetic.

Worked examples

Substitute:

```
echo "foo bar foo" | sed 's/foo/X/'
# sample output: X bar foo
echo "foo bar foo" | sed 's/foo/X/g'
# sample output: X bar X
```

Delete and select lines:

```
seq 1 5 | sed -n '2,4p'
# sample output: 2
# sample output: 3
# sample output: 4
printf 'a\n\nb\n' | sed '/^$/d'
# sample output: a
# sample output: b
```

In-place edit with backup:

```
echo "port=80" > app.conf
sed -i.bak 's/^port=.*/port=8080/' app.conf
cat app.conf app.conf.bak
# sample output: port=8080
# sample output: port=80
```

Columns with awk:

```
printf 'a 10\nb 20\nc 5\n' > data.txt
awk '{print $1}' data.txt
# sample output: a
# sample output: b
# sample output: c
awk '{s += $2} END {print s}' data.txt
# sample output: 35
```

Filter by condition and format:

```
awk '$2 > 8 {printf "%s is big (%d)\n", $1, $2}' data.txt
# sample output: a is big (10)
# sample output: b is big (20)
awk -F: '$3 == 0 {print $1}' /etc/passwd
# sample output: root
```

Average using NR:

```
awk '{s += $2} END {print s / NR}' data.txt
# sample output: 11.6667
```

Mini-lab

1. `cd /tmp && printf 'apple 3\npear 5\nfig 2\n' > fruit.txt`.
2. Run `awk '{print $2}' fruit.txt` and `awk '{s+=$2} END{print s}' fruit.txt` (prints 10).
3. Run `sed 's/ /,/' fruit.txt` and compare with `sed 's/a/A/g'`.
4. Run `sed -n '/pear/p' fruit.txt` and `sed '/fig/d' fruit.txt`.
5. Run `awk 'NR==2 {print $1}' fruit.txt`.
6. Create `cp fruit.txt f2.txt && sed -i.bak 's/pear/plum/' f2.txt` and `diff f2.txt f2.txt.bak`.
7. Use `ps aux | awk '{print $1}' | sort | uniq -c` to count processes per user.
8. Use `awk -F: '{print $1 ":" $7}' /etc/passwd | head -n 3`.

Decision guide / failure modes

Use `sed` for find-and-replace and line ranges; use `awk` when you need columns, conditions, sums or formatted output; use `cut` for simple fixed-column extraction. For structured formats (JSON, real CSV with quotes) use `jq` or a CSV tool instead; awk and sed will break on embedded separators.

Common failures:

- `sed -i` on macOS needs `-i ''`; GNU and BSD differ. Test on your target system.
- Using `/` in a pattern that contains paths; choose another delimiter: `s|/usr/local|/opt|`.
- Shell quoting: use single quotes so `$1` reaches awk instead of the shell.
- Non-numeric fields (a header row) counted as 0 in sums; skip with `NR>1`.
- Greedy `.*` matching more than expected.
- Editing files in place with no backup; try without `-i` first.

Common mistakes

- Blind in-place sed without backup
- Using awk where `cut` suffices (or vice versa for complex fields)
- Assuming GNU sed flags on BSD/macOS

Exercises

Work in `exercises/09-sed-awk/`. Read that folder's `README.md` fully before coding.

```
cd exercises/09-sed-awk
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement sum_field2(text). The input is multi-line text where each line has whitespace-separated fields. Return the sum of the second field ($2 in awk) across all lines, as a number (int when all values are integers is fine, or float). Lines with fewer than two fields are skipped, as are lines whose second field is not a number (like a header). Blank lines are skipped. An empty input gives 0. This is the Python equivalent of awk '{s += $2} END {print s}' with a little extra safety for headers and short lines.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/09-sed-awk/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What does `s/a/b/g` mean?
2. How set awk field separator to `:`?
3. When prefer awk over cut?

Further reading

- https://www.gnu.org/software/gawk/manual/